

Compare This Model to Your Earlier Two-Layer Neural Network Response:

In Module 12, the two-layer neural network was limited to using only a small set of structured features such as GPA, GRE scores, degree type, and citizenship. In contrast, the PyTorch transformer model can incorporate both structured data and unstructured text, including applicant comments and program names, allowing it to capture richer contextual information. Including text fields such as comments appears to provide additional signals, especially when applicants describe strong research experience or academic background, which the earlier model could not interpret. The pretrained transformer model is more flexible because it can understand language patterns and relationships, but it can also be more fragile due to sensitivity to noisy, incomplete, or inconsistent text data. Overall, the added complexity feels justified, as the model achieves stronger performance and better captures real-world nuances, although the quality of the dataset still limits how reliable the predictions can be.

Reflection on Limitations and Ethics Response:

Fine-tuning a pretrained language model is valuable because it leverages patterns learned from large-scale text data, allowing the model to understand language and context without needing to train from scratch. Combining text and structured features is particularly useful in this task, as admissions decisions often depend not only on numeric metrics like GPA and GRE scores but also on qualitative information such as applicant background and experiences. However, the Grad Cafe dataset introduces significant bias because it is self-reported, incomplete, and likely skewed toward certain types of applicants, which can lead to misleading patterns. As a result, the model may produce unfair or inaccurate predictions, especially when key factors such as recommendation letters, personal statements, institutional priorities, and holistic review criteria are missing. A model like this should not be used for real admissions decision-making, as it lacks critical context and could reinforce biases. Instead, uncertainty should be clearly communicated to users, and disclaimers should emphasize that predictions are approximate and not authoritative. Despite these limitations, a project like “Will You Get In?” is educationally valuable because it demonstrates how modern machine learning models can integrate diverse data types and highlights both the power and risks of applying predictive models to complex real-world problems.

Preprocessing decisions:

```
1. LOAD AND PREPARE THE APPLICANT DATASET
Number of rows in the original dataset: 28631
*** Number of rows remaining after filtering: 21274
Number of Accepted rows: 9706
Number of Rejected rows: 11568

Fields used for modeling:
- program_name
- comments
- llm-generated-program
- llm-generated-university
- start_term
- gpa
- gre_score
- gre_v_score
- gre_aw
- degree
- international_american

Preview of cleaned dataframe:
      program_name  ... label
0  Biomedical Engineering  ...    0
1  Sociocultural Anthropology  ...    0
2    Public Administration  ...    0
3  Health Promotion, Education, and Behavior  ...    0
4    Clinical Psychology  ...    1
```

The dataset was preprocessed by filtering for Accepted and Rejected outcomes, removing duplicate entries based on the application URL, and converting numeric fields such as GPA and GRE scores to numeric format. Missing values were handled using consistent placeholders such as “Unknown” or “None” to ensure all inputs remained usable. Text fields and categorical variables such as degree and citizenship were normalized, and rows without sufficient usable information were removed.

Model input template:

2. CONVERT EACH APPLICANT INTO A UNIFIED MODEL INPUT

```
*** Exact template used:
Program: {program_name}
LLM Program: {llm-generated-program}
University: {llm-generated-university}
Comments: {comments}
Start Term: {start_term}
Degree: {degree}
Citizenship: {international_american}
GPA: {gpa}
GRE Total: {gre_score}
GRE Verbal: {gre_v_score}
GRE AW: {gre_aw}
```

Three sample model inputs:

```
--- SAMPLE INPUT 1 ---
Program: Economics
LLM Program: Economics
University: University of Milan
Comments: None
Start Term: Fall 2025
Degree: PhD
Citizenship: International
GPA: Unknown
GRE Total: 3200
```

Each applicant was converted into a unified text representation combining both structured and unstructured features. The input template included program name, LLM-generated program and university, comments, start term, degree type, citizenship, GPA, and GRE scores. This allowed the transformer model to process all information as a single sequence of text.

Training configuration:

```
* 4. FINE-TUNE A PRETRAINED PYTORCH LANGUAGE MODEL
Chosen model: distilbert-base-uncased
Why this model: DistilBERT is lightweight, fast, and practical for class assignments.
Tokenizer name: distilbert-base-uncased
Max sequence length: 256
Batch size: 8
Number of epochs: 2
Learning rate: 2e-05
Optimizer used: AdamW (inside Hugging Face Trainer)
Device used: cuda
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100% 100/100 [00:00<00:00, 2326.85it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key | Status |
-----+-----+
vocab_layer_norm.bias | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
vocab_transform.weight | UNEXPECTED |
vocab_projector.bias | UNEXPECTED |
vocab_transform.bias | UNEXPECTED |
pre_classifier.bias | MISSING |
classifier.bias | MISSING |
pre_classifier.weight | MISSING |
classifier.weight | MISSING |
```

The model was fine-tuned using the DistilBERT pretrained transformer model. Training was performed with a maximum sequence length of 256, batch size of 8, and 2 training epochs. A

learning rate of $2e-5$ was used along with the AdamW optimizer through the Hugging Face Trainer API. Training was conducted on a GPU-enabled environment using Google Colab.

Evaluation metrics:

5. EVALUATE THE FINAL MODEL

Metrics summary:

Accuracy: 0.7885

Precision: 0.7809

Recall: 0.7455

F1: 0.7628

Confusion matrix:

```
[[1908  406]
 [ 494 1447]]
```

Class distribution in test set:

0 2314

1 1941

Name: count, dtype: int64

Probability examples for several predictions:

Example 1: True=1, Pred=1, P(Rejected)=0.0114, P(Accepted)=0.9886

Model performance was evaluated using accuracy, precision, recall, and F1 score. The final model achieved approximately 78% accuracy, with balanced precision and recall, indicating that it performs well across both Accepted and Rejected classes.